

Chapter3:Logical Database Design

Database System Concepts

University of Guilan

Yasaman Boreshban

Fall 2025

Outline

- Introduction to logical database design concepts
- Data structures and the Table Data Structure (TDS) model
- Definition of keys and relational integrity
- Logical design of 1:1, 1:N, and M:N relationships
- Identifying IS-A relationships
- Multivalued attributes
- Practical exercises on TDB design

Logical Database Design

- Data modeling can occur at different levels of abstraction.
- One level below conceptual data modeling is the **logical design level**.
- At this level, we focus on how data will be represented in the database using models that are **independent of physical implementation**.
- Levels of Abstraction:
 - 1. Real-world understanding
 - 2. Conceptual data modeling
 - 3. Logical database design

Database Design (continued)

- Introductory discussion: Focus on a practical rather than theoretical perspective.
- For the logical design of databases (and also for operations and control in DB systems),
 - it is essential to define a specific framework known as a **Data Model (DM)**,
 - which includes a **Data Structure (DS)**.
- Concepts used in logical database design:
 - TDS : Table Data Structure
 - TDB : Table Database
 - TDBL : Table Database Language

Data Structures

- Why Data Structures (in general)?
- To represent the types of entities and the relationships between them
 - at the **logical** and **physical** levels of database design.
- Reasons for the necessity of Data Structures (DS) in database context:
 - Provides the environment for files (abstract environment).
 - Serves as the framework for logical database design.
 - Forms the basis for the design of database languages (DBL).
 - Forms the basis for the design of DBMS itself.
 - Provides metrics for comparing and evaluating systems.
 - Provides a foundation for developing and extending database design techniques.
 - 7. ...

Data Structures (continued)

- DSs in the domain of database knowledge and technology:
 - HDS → Hierarchical Data Model (HDBMS)
 - NDS → Network Data Model (NDBMS)
 - RDS → Relational Data Model (RDBMS)
 - ODS → Object Data Model (ODBMS)
 - ORDS → Object-Relational Data Model (ORDBMS)
- TDS – Table Data Structure:
 - The fundamental structural element in the Relational Model (RM).
 - A Relation is a mathematical concept represented as a table.
 - Each relation consists of rows (tuples) and columns (attributes).
 - In practice, this tabular representation is implemented through TDS.
 - Therefore, RDS is based on TDS.

TDS Terminology:

- Table Type : represents the type of entities and/or relationships.
 - (Table name + names and types of columns)
- Row (Tuple) : represents an instance of an entity or relationship.
- Column (Attribute) : represents a property or characteristic.
- Note:
 - The identifying attribute of entities plays the role of the **Key** in the table.

Table Database (TDB)

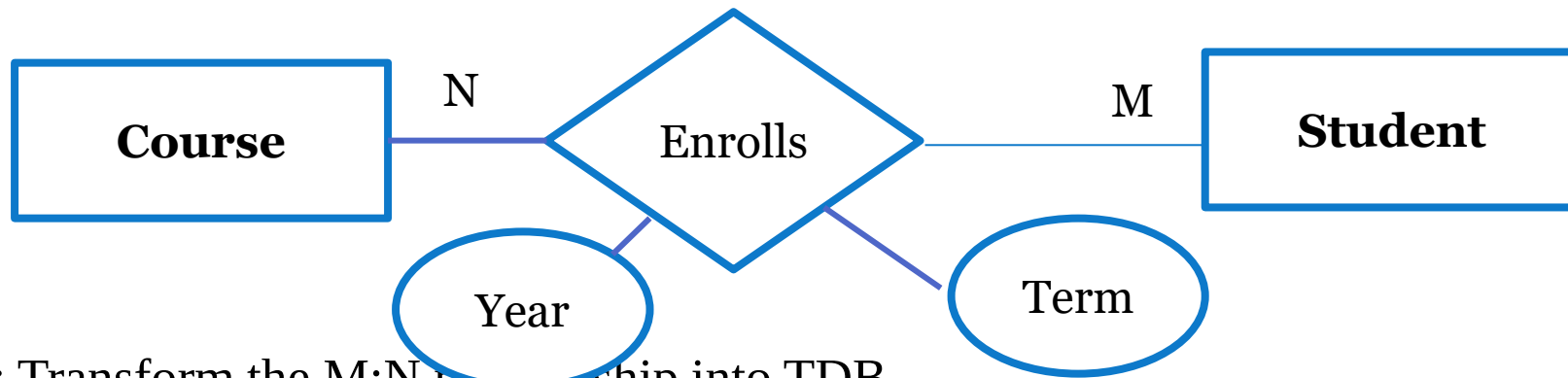
- From a user's point of view:
 - Database Designer (DB designer)
 - Database Implementer (DB builder)
 - Application Programmer (AP developer)
- From a structural perspective:
 - A TDB consists of a collection of **Table Types** (which we design).
- From a data content perspective:
 - A TDB consists of a collection of **distinct table instances**.
 - Each table instance contains rows (tuples) representing entity or relationship data.

Key Concept

- From an application perspective:
 - A **key** provides access to a single instance (of an entity or relationship).
 - Therefore, a key uniquely identifies a record in a table .it ensures uniqueness.
- From a design perspective:
 - One or more columns (attributes) together form the **primary key**,
 - provided that their values are both **unique** and **known** for all rows.
- Sometimes, more than one candidate key may exist.
 - One of them is chosen as the **primary key**,
 - while the others are declared as **unique** (as in SQL).

Example: M:N Relationship

- To determine cardinality, analyze both directions



- Problem: Transform the M:N relationship into TDB
 - For each **entity type**, we need one **table type**.
 - For each **M:N relationship**, we also need one **table type**.
- Therefore, in this example, three table types are required:
 - STUDENT table
 - COURSE table
 - ENROLLMENT table

Logical Design with TDS – Many-to-Many Relationship (continued)

STT	<u>STID</u>	STNAME	STLEV	STMJR	STDEID
	777	st7	bs	phys	d11
	888	st8	ms	math	d12
	444	st4	ms	phys	d11
	⋮	⋮	⋮	⋮	⋮
COT	<u>COID</u>	COTITLE	CREDIT	COTYPE	CEDEID
	⋮	⋮	⋮	⋮	⋮
	co3	programming	4	t	d13
	⋮	⋮	⋮	⋮	⋮

- The underlined attribute represents the **Primary Key** of the table.
- Each table (TDS) has one key that uniquely identifies its rows.

Logical Design with TDS – Many-to-Many Relationship (continued)

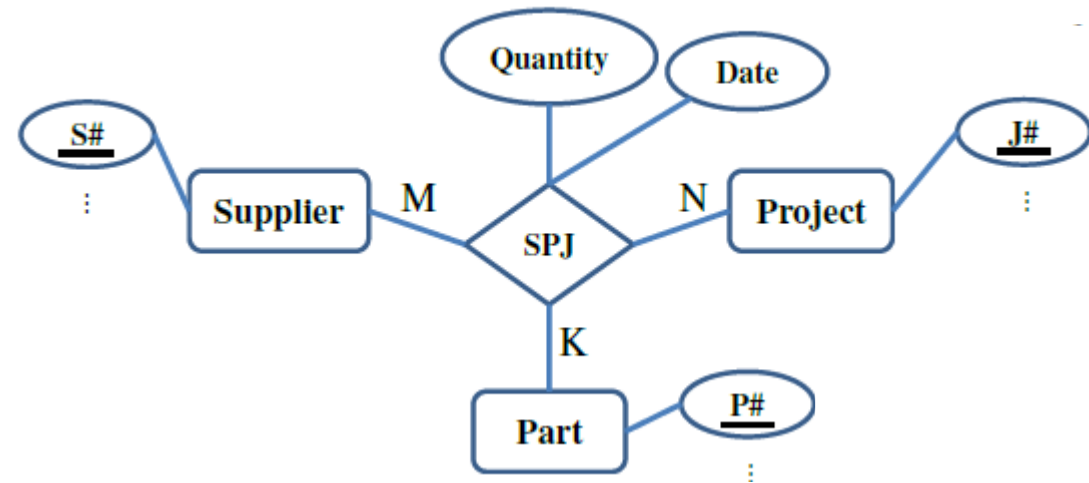
- According to semantic rules, attributes such as "Year" and "Term"
- may also be part of the key, depending on the environment.



<u>STID</u>	<u>COID</u>	<u>TR</u>	<u>YR</u>
⋮	⋮	⋮	⋮
888	co2	1	87
888	co3	1	87
444	co2	1	87

- Note:
 - In some cases, a key can be composite, consisting of multiple attributes.
 - Here, (STID, COID, TR, YR) together form a composite key for the M:N relationship table.

Logical Design with TDS – Many-to-Many Relationship (continued)



- For each **entity type**, one **table type** is required.
- For each **relationship type**, one **table type** is also required.
- Therefore, we need four table types:
 - SUPPLIER
 - PART
 - PROJECT
 - SPJ (the relationship table)

Logical Design– Many-to-Many Relationship (continued)

- According to semantic rules, the "Date" attribute may also be part of the key.
- If a record has multiple quantitative attributes, the key becomes **composite**.

Supplier	<u>S#</u>	SNAME	CITY	...
	s1	...	c1	...
	s2	...	c1	...
	⋮	⋮	⋮	⋮

Part	<u>P#</u>	PNAME	CITY	...
	p1	...	c1	...
	p2	...	c2	...
	⋮	⋮	⋮	⋮

Project	<u>J#</u>	JNAME	CITY	...
	j1	...	c2	...
	j2	...	c1	...
	⋮	⋮	⋮	⋮

SPJ	<u>S#</u>	<u>P#</u>	<u>J#</u>	<u>Date</u>	QTY
	s1	p1	j1	d1	100
	s1	p1	j1	d2	50
	⋮	⋮	⋮	⋮	⋮



Logical Design with TDS – One-to-Many Relationship

- Problem: Transform the 1:N relationship into TDB (or TDS).



- We need two table types:
 - One table for the entity type on the 1” side (TEACHER)
 - One table for the entity type on the N” side (GROUP),
 - which also includes the foreign key representing the relationship.

Logical Design with TDS – One-to-Many Relationship (continued)

DEPT	<u>DEID</u>	DETITLE	...	DEPHONE
	D11	Phys	...	...
	D12	Math	...	...
	⋮	⋮	⋮	⋮

PROF	<u>PRID</u>	PRNAME	RANK	...	FROM	<u>DEID</u>
	Pr100	...	استاد	...	d1	D13
	Pr200	...	استادیار	...	d2	D11
	Pr300	...	دانشیار	...	?	?

- The column DEID in the PROFESSOR table is a foreign key, indicated by underlined italics.
- foreign key (FK) in table T2 refers to the primary key (PK) of table T1.
- The relationship between the two tables represents the 1:N relationship between Department and Professor.
- Each department (DEPT) may have multiple professors (PROF).

Logical Design with TDS – One-to-One Relationship

- Example: 1:1 Relationship
- Both sides have a cardinality of 1, representing a one-to-one relationship.



- We need two table types:
 - One table for the entity type on the “1” side (GROUP)
 - One table for the entity type on “1” side (TEACHER),
 - which also includes the foreign key representing the relationship.
- This relationship indicates that each teacher manages at most one group, and each group is managed by at most one teacher.
- The relationship can be implemented by placing a foreign key in either table.
- Usually, the foreign key is placed in the table corresponding to the non-mandatory side.

Logical Design with TDS – One-to-One Relationship (continued)

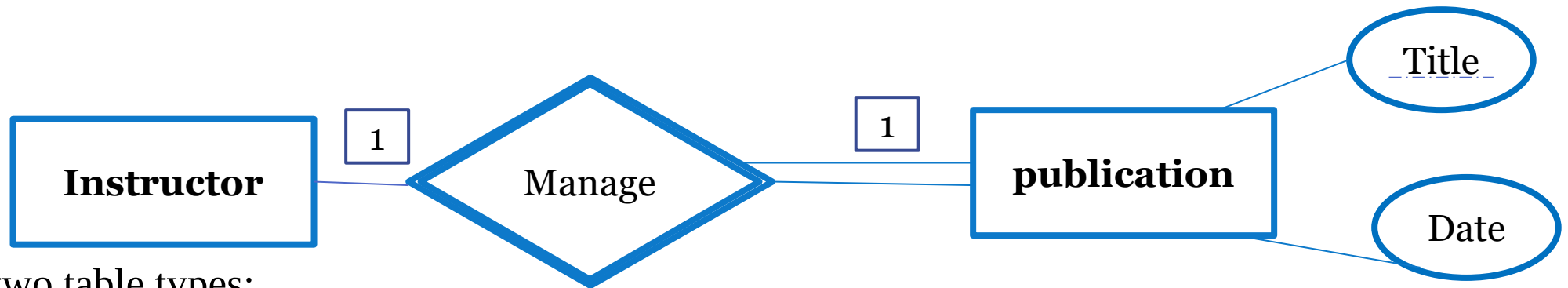
DEPT	<u>DEID</u>	DETITLE	...	DEPHONE	<u>PRID</u>
	D11	Phys	...	...	...
	D12	Math	...	...	...
	⋮	⋮	⋮	⋮	⋮

PROF	<u>PRID</u>	PRNAME	RANK	...
	Pr100	...	استاد	...
	Pr200	...	استادیار	...
	Pr300	...	دانشیار	...
	⋮	⋮	⋮	⋮

- The column PRID in the DEPARTMENT table is a **foreign key** referencing the PROFESSOR table.
- This represents a one-to-one relationship between DEPARTMENT and PROFESSOR.
- Each department has at most one professor as its head (manager).
- Alternatively, the foreign key could be placed in the PROFESSOR table, depending on which side is optional.

Logical Design with TDS – Identifying Relationship

- Identifying Relationship (Weak Entity Relationship)



- We need two table types:
 - One table for the strong entity type (Professor)
 - One table for the weak entity type (Published Work),
 - which includes the identifying relationship and contains the primary key of the strong entity (as a foreign key).
- The identifying relationship ensures that the weak entity (Published Work)
- cannot exist without its owner (Professor).
- The primary key of the weak entity includes the primary key of the strong entity.

Logical Design with TDS – Identifying Relationship (continued)

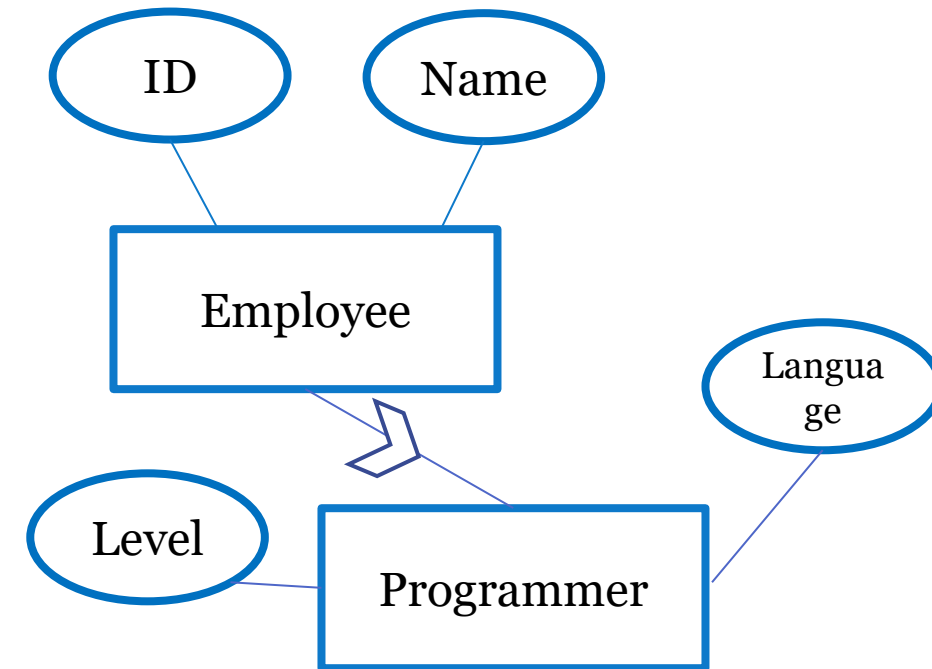
PROF	<u>PRID</u>	PRNAME	RANK	...
	Pr100	...	استاد	...
	Pr200	...	استادیار	...
	Pr300	...	دانشیار	...
	⋮	⋮	⋮	⋮

PUB	<u>PRID</u>	PTITLE	...	PDATE
	Pr100	Data Encryption...	...	...
	Pr100	Semantic Analysis of ...	...	...
	⋮	⋮	⋮	⋮

- The attributes PRID (foreign key from PROFESSOR) and PTITLE together form the primary key of the PUBLICATION table.
- This shows an identifying relationship between PROFESSOR (strong entity) and PUBLICATION (weak entity).
- If a professor is deleted, all their related publications should also be deleted (cascading delete).
- Updates in the PROFESSOR table should also maintain referential integrity with PUBLICATION.

Logical Design with TDS – IS-A Relationship

- IS-A Relationship Example:
- We need two table types:
 - One table for the supertype (Employee),
 - containing common or shared attributes.
 - One table for the subtype (Programmer),
 - containing specific attributes of the subtype and the identifier (foreign key) referencing the supertype.



- Each subtype (Programmer) inherits the attributes of its supertype (Employee).
- The primary key of the subtype table is also a foreign key referencing the supertype.
- This structure maintains the IS-A hierarchy in the relational model.

Logical Design with TDS – IS-A Relationship (continued)

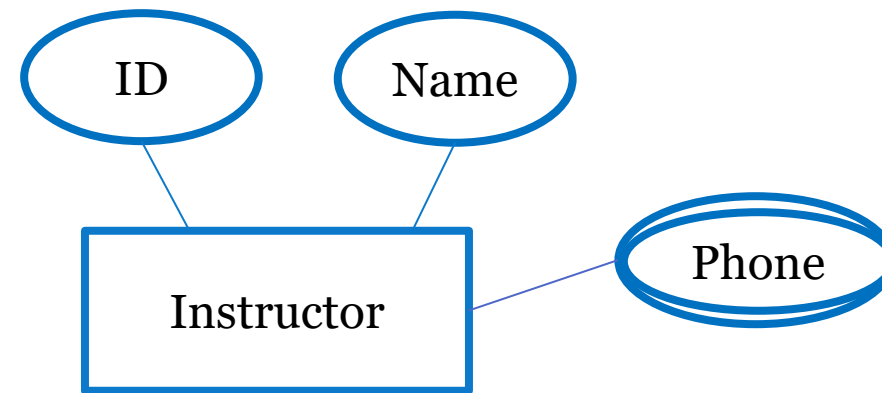
EMP	<u>EID</u>	ENAME	EBDATE	...	EPHONE
	E100	...	...	...	...
	E101	...	...	...	...
	E102	...	...	...	...
	⋮	⋮	⋮	⋮	⋮

PROG	<u>EID</u>	LANG	...	LEVEL
	E100	C++	...	...
	E102	Java	...	...
	⋮	⋮	⋮	⋮

- The attribute EID is a foreign key referencing EMP (Employee) and also serves as the primary key of the PROG (Programmer) table.
- This enforces the IS-A relationship, ensuring that each Programmer is an Employee.
- If an EMP record is deleted or updated, corresponding changes must cascade to PROG to maintain referential integrity (and vice versa, as appropriate).

Logical Design with TDS – Multivalued Attribute

- Multivalued Attribute Example:



- We need two tables:
 - One table for the entity type (Professor) ,containing single-valued attributes.
 - One table for the multivalued attribute (Phone), containing the entity key and the multivalued attribute.
- Each professor can have one or more phone numbers.
- A separate table is created for the multivalued attribute to ensure normalization.
- The table for Phone will include a foreign key referencing the Professor table.

Logical Design with TDS – Multivalued Attribute (continued)

PROF	<u>PRID</u>	PNAME	RANK	...	...
	Pr100	...	...	...	...
	Pr101	...	...	...	...
	Pr102	...	...	...	...
	⋮	⋮	⋮	⋮	⋮

PROFTEL	<u>PRID</u>	<u>TEL</u>
	Pr100	09121234567
	Pr100	02177889911
	Pr101	09352348762
	⋮	⋮

- The attribute TEL is a multivalued attribute for the entity PROFESSOR.
- Each professor may have several phone numbers, so the multivalued attribute is stored in a separate table (PROFTEL).
- PRID is a foreign key referencing the primary key of the PROF table.
- The combination (PRID, TEL) forms the composite primary key for the PROFTEL table.
- This structure avoids data redundancy and maintains normalization.

End of Chapter

